PCCCS495: Advanced Programming (OOP) Lab

Term-II Individual Project Report

Smart Library Management System

Student Name: Saraswata Tarafdar

Roll No.: 62

Instructor Name: Soumadip Biswas, Susovan Jana, Swagatam Basu

Semester: 4th

Spring 2026

1. Problem Statement

Managing a library manually is time-consuming and error-prone. Librarians struggle to track which books are available, which are borrowed, who borrowed them, and when they are due. There is no easy way to search for a book, record fines for late returns, or keep member records organised. Duplicate borrowing, lost return records, and untracked overdue fines are common problems.
This project provides a structured object-oriented Java application — the Smart Library Management System — that handles book inventory, member management, borrowing and return operations, and automatic fine calculation in a reliable and extensible way.
Target Users:

- Librarians who manage books and magazines
- Library members who borrow and return items
- Administrators who manage records and fines

2. System Design

Package Structure:

model — LibraryItem, Book, Magazine, Member, BorrowRecord
Core entity classes

exception — BookNotFoundException, AlreadyBorrowedException, MemberNotFoundException
Custom error handling

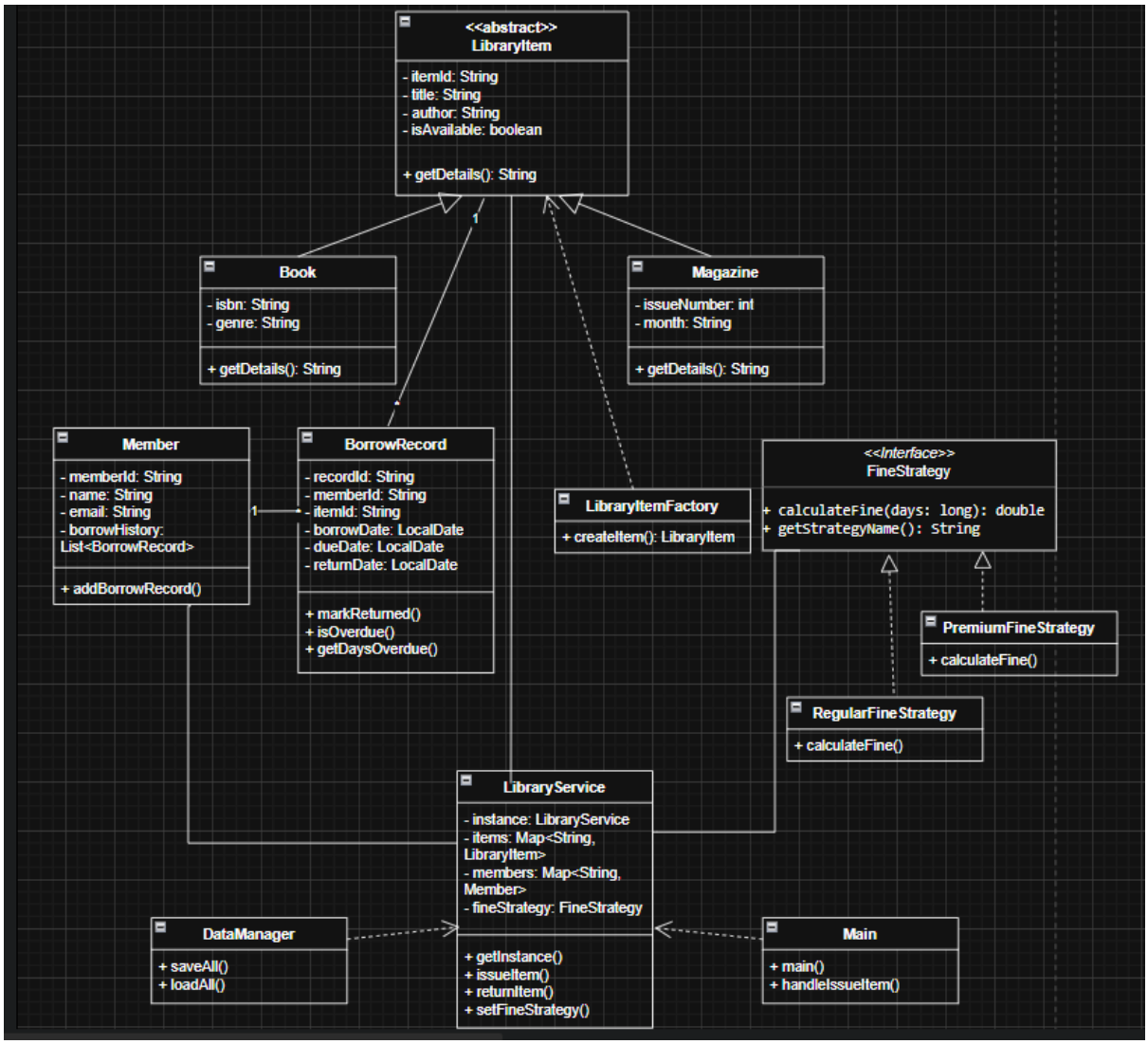
factory — LibraryItemFactory
Object creation

strategy — FineStrategy, RegularFineStrategy, PremiumFineStrategy
Fine calculation logic

service — LibraryService, DataManager
Business logic and data persistence

ui — Main
Console interface

UML Diagram:



UML Class Overview:

LibraryItem (abstract)

- itemId, title, author, isAvailable
- getDetails()

Book extends LibraryItem

- isbn, genre

Magazine extends LibraryItem

- issueNumber, month

Member

- memberId, name, email
- borrowHistory

BorrowRecord

- recordId, memberId, itemId
- borrowDate, dueDate, returnDate

FineStrategy (interface)

- calculateFine(days)

LibraryService (Singleton)

- manages items, members, and operations

LibraryItemFactory

- creates objects

DataManager

- handles file storage

Interaction Flow:

Issue Item Process:
Main → LibraryService.issueItem()
→ validate member
→ validate item
→ check availability
→ create BorrowRecord
→ update item and member
→ save data

3. OOP Concepts Demonstrated

3.1 Abstraction

Where used: LibraryItem.java (model package)

Why used: A bare 'LibraryItem' object makes no sense on its own — the library only ever holds Books or Magazines. Making LibraryItem abstract forces every subclass to implement getDetails(), ensuring the method always exists but the behaviour is defined by the concrete type.
Explanation: The abstract keyword prevents instantiation of LibraryItem directly. The getDetails() abstract method is a contract — every subclass must provide its own version.

3.2 Inheritance

Where used: Book.java and Magazine.java (model package)

Why used: Both Book and Magazine share common properties (itemId, title, author, isAvailable) and common behaviour (getters, setAvailable, toString). Inheritance eliminates duplication — these fields and methods are written once in LibraryItem and reused in both subclasses.

Explanation: super(itemId, title, author) must be the first line — it initialises the parent class fields before Book's own fields are set.

3.3 Polymorphism

Where used: getDetails() method in Book and Magazine

Why used: Both Book and Magazine override getDetails(), but produce different formatted output. This allows a List<LibraryItem> to store both types, and calling getDetails() on any element automatically calls the correct version without any type checking.

Explanation: Same method name, different behaviour depending on the actual object type at runtime — this is runtime polymorphism.

3.4 Exception Handling

Where used: exception package, LibraryService.java

Why used: Generic errors give no useful information. Custom exceptions like BookNotFoundException carry the specific item ID that was not found, making debugging and user feedback precise.

Explanation: extends Exception makes it a checked exception — Java forces the caller to handle it with try-catch. This prevents the program from crashing silently on invalid input.

3.5 Collections

Where used: LibraryService.java (HashMap), Member.java (ArrayList)

Why used: HashMap provides O(1) key-based lookup — finding a book by ID is instant regardless of how many books exist. ArrayList in Member stores the full borrowing history in insertion order.

4. Implementation Highlights

4.1 Singleton Pattern

LibraryService uses the Singleton design pattern to ensure only one instance of the service exists throughout the program's lifetime. If two instances existed, they would hold separate HashMaps — books added in one would not appear in the other, causing data inconsistency.

4.2 Factory Pattern

The Factory Pattern centralises object creation. Instead of calling new Book(...) or new Magazine(...) scattered across the codebase, all creation goes through LibraryItemFactory.createItem(). If a constructor ever changes, only the factory needs updating.

4.3 Strategy Pattern

The Strategy Pattern allows the fine calculation algorithm to be swapped at runtime without modifying LibraryService. This directly addresses the instructor's feedback requesting an additional design pattern.

4.4Java Serialization

All library data is persisted to disk using Java Object Serialization, storing the complete HashMap of items and members in binary .ser files that survive program restarts.

5. Testing and Error Handling

5.1 Test Coverage

Includes 16 JUnit test cases covering:

- Adding/removing items
- Searching items
- Member management
- Borrow/return logic
- Fine calculations
- Exception handling

5.2 Edge Cases

- Zero or negative overdue → no fine
- Invalid inputs handled safely
- Missing files → system initializes cleanly
- Invalid menu input → handled without crash

5.3 Failure Scenarios

- Borrowing unavailable item → handled
- Invalid member lookup → handled
- Missing item removal → handled
- Corrupt data files → handled

6. Git Workflow

Development Timeline:

Commit 1 — Initialize project structure with package folders
Commit 2 — Book class extending LibraryItem
Commit 3 — Magazine class extending LibraryItem
Commit 4 — Member Class and BorrowRecord
Commit 5 — Custom Exceptions
Commit 6 — LibraryItemFactory using Factory design pattern
Commit 7 — Adding Fine Strategies
Commit 8 — LibraryService with Singleton pattern and HashMap storage
Commit 9 — DataManager with Java Serialization
Commit 10 — Main file with console menu and all user options
Commit 11 – Testing code and Documentation

Commit Discipline:

- Each commit represented a single logical unit of work — one class or one feature per commit
- Commit messages were descriptive and action-oriented (verb-first format)
- No large bulk commits — code was pushed daily as it was written.

7. Conclusion and Future Scope

The Smart Library Management System successfully demonstrates all core OOP concepts required by the course — abstraction, inheritance, polymorphism, encapsulation, exception handling, and collections. It additionally implements three design patterns (Singleton, Factory, Strategy), uses Java Serialization for persistence, and includes a full JUnit test suite. The project was deliberately kept focused and well-designed rather than large and feature-heavy, in line with the course philosophy that design maturity matters more than feature count.

- GUI layer using Java Swing with event-driven architecture (as suggested by the instructor)
- Observer Pattern to notify members when a reserved book becomes available
- Database persistence using JDBC instead of flat .ser files for concurrent access
- Student member type with a different fine cap and longer borrow period
- Search by ISBN for books and by issue number for magazines

Appendix

A. Project Proposal

1. Project Title

Smart Library Management System

2. Problem Statement

Managing a library manually is time-consuming and error-prone. Librarians struggle to track which books are available, which are borrowed, who borrowed them, and when they are due. There is no easy way to search for a book, record fines for late returns, or keep member records organized. This system solves these problems by providing a well-structured, object-oriented Java application that handles book inventory, member management, and borrowing/return operations in a clean and reliable way.

3. Target User

- Librarians who need to manage book inventory and member records
- Library members who borrow and return books
- Administrators who oversee the overall system

4. Core Features

- Add, remove, and search books by title, author, or ISBN
- Register and manage library members
- Issue (borrow) books to members with due date tracking
- Return books and automatically calculate overdue fines
- View borrowing history for any member
- Exception handling for invalid operations (e.g. borrowing an already-borrowed book)

- File-based persistence to save and load library data

5. OOP Concepts to be Used

Abstraction

- Abstract class LibraryItem with abstract method getDetails() — extended by Book and Magazine

Inheritance

- Book and Magazine both extend LibraryItem, sharing common fields like title, itemId, and availability status

Polymorphism

- getDetails() method behaves differently for Book vs Magazine — called through a LibraryItem reference

Exception Handling

- Custom exceptions: BookNotFoundException, AlreadyBorrowedException, MemberNotFoundException

Collections

- ArrayList and HashMap used to store books, members, and borrow records

6. Proposed Architecture

The project will follow a clean layered package structure. The model package contains core entity classes (LibraryItem, Book, Magazine, Member, BorrowRecord). The service package contains the LibraryService class which handles all business logic such as issuing books, returning books, and calculating fines. The exception package contains all custom exception classes. The ui package contains the Main class which provides a simple console-based menu for interaction. Data will be saved and loaded using file handling (text files or serialization) to persist state between runs. At least one design pattern will be used — the Singleton pattern for the LibraryService to ensure only one instance manages the library state.

B. Proposal Review

Project: Smart Library Management System
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal clearly defines the problem, target audience, and a comprehensive set of features for a practical library management system. It explicitly lists and plans to implement core OOP concepts (Abstraction, Inheritance, Polymorphism, Exception Handling, Collections), which is excellent for a university OOP project. The layered architecture is well-defined and logical, separating concerns effectively. The inclusion of file-based persistence is crucial for a stateful application, and the commitment to using a design pattern

(Singleton for LibraryService) demonstrates thoughtful design considerations.

Improvements:
While file-based persistence is good, specifying the exact mechanism (e.g., Java Serialization for objects or specific text file formats) would provide more clarity. The proposal would benefit from including a plan for unit testing to ensure the robustness and correctness of the business logic. Although a console UI is acceptable for a learning project, mentioning the potential for a simple graphical user interface (GUI) as an extension could add another dimension to the OOP learning experience (e.g., event handling, UI component hierarchies). Exploring one or two additional design patterns (e.g., Factory for creating different types of LibraryItems or Strategy for fine calculation) could further enhance the architectural maturity.

Regards,
Course Instructor